

Paws Rescue Donations

A Line-by-Line Deep Dive

Prepared for Salman Adnan — reference document, not published content

Compiled July 2026

How to read this document

This deep dive assumes no prior Django or web-framework knowledge. Every technical term is defined in a highlighted box the first time it appears. Diagrams are used wherever a picture explains the structure faster than prose. Everything stated here was verified directly against the project's source code, templates, migrations, and README as of the date this document was compiled – nothing is invented or estimated.

Contents

1	What This Project Is	3
1.1	Framework: Django	3
2	Directory Structure	3
3	Architecture Overview	4
4	Request Lifecycle and URL Routing	5
4.1	Full URL Table	6
5	Data Models and Entity Relationships	7
6	The accounts App	7
6.1	Custom User Model (accounts/models.py)	7
6.2	Login Form and View	9
6.3	Admin Management Views	10
6.4	Templates	10
6.5	Tests (accounts/tests.py)	11
7	The donations App	11
7.1	The Donation Model	11
7.2	Forms (donations/forms.py)	12
7.3	The Dashboard View: Filtering, Analytics, Pagination	12
7.4	CSV Export and the Session-Based Filter Handoff	14
7.5	Donation Create / Update / Delete Views	15
7.6	Templates and Template Inheritance	15
7.7	Tests (donations/tests.py)	15
8	The Management Command: create_initial_admin	16

9 Configuration: settings.py	16
10 Dependencies (requirements.txt)	18
11 Deployment (Procfile)	18
12 Static Assets	19
13 Environment Variable Surface (.env.example)	19
14 Known Limitations (as documented by the project itself)	19
Appendix: Glossary	20

1 What This Project Is

Paws Rescue Donations is a small, single-purpose web application: it lets the staff of a pet rescue organization log in and record, browse, filter, and export donation records. There is no public-facing part of the site at all – every page except the login screen requires an authenticated staff account. It was built as a faculty-supervised community-service (“Khidmat”) project at Habib University, and is deliberately scoped to be simple enough to run for free on a small hosting tier.

Web application. A program that runs on a server and is used through a web browser, as opposed to an app installed directly on a phone or desktop. The **server** runs the code and stores the data; the browser only renders HTML pages sent to it.

The problem it solves, in the project’s own words (from its README): small rescues usually track donations in a spreadsheet, which breaks down once multiple people enter records – there is no access control, no consistent set of fields, and no easy way to answer a question like “how much has Bella received this month?” This app replaces the spreadsheet with a database and a login.

1.1 Framework: Django

The project is built with **Django** 4.2, a Python web framework.

Framework. A pre-built skeleton of code that handles the repetitive parts of a category of program (here: web applications) so the developer only writes the parts specific to their app. Django supplies the web server glue, the database layer, the templating engine, and the authentication system; this project supplies the rescue-specific models, views, and templates on top of it.

MVT (Model–View–Template). Django’s variant of the well-known Model–View–Controller (MVC) pattern. A *Model* defines the shape of data and talks to the database. A *View* is a Python function (or class) that receives a web request, decides what data is needed, and returns a response. A *Template* is an HTML file with placeholders that the view fills in with data before sending it to the browser. In classic MVC terms, Django’s “View” plays the role of the *Controller*, and its “Template” plays the role of the *View*. This is a common source of confusion for people coming from other frameworks.

2 Directory Structure

The diagram below shows every file in the repository and where it sits. Folders are in bold; leaf nodes are files.

Django app. A self-contained module of related functionality inside a Django project – its own models, views, URLs, and templates. A single Django *project* (here, `paws_rescue`) can contain many apps; this project has two, `accounts` and `donations`, both nested under a container package named `apps` purely for import-path tidiness (`apps.accounts`, `apps.donations`).

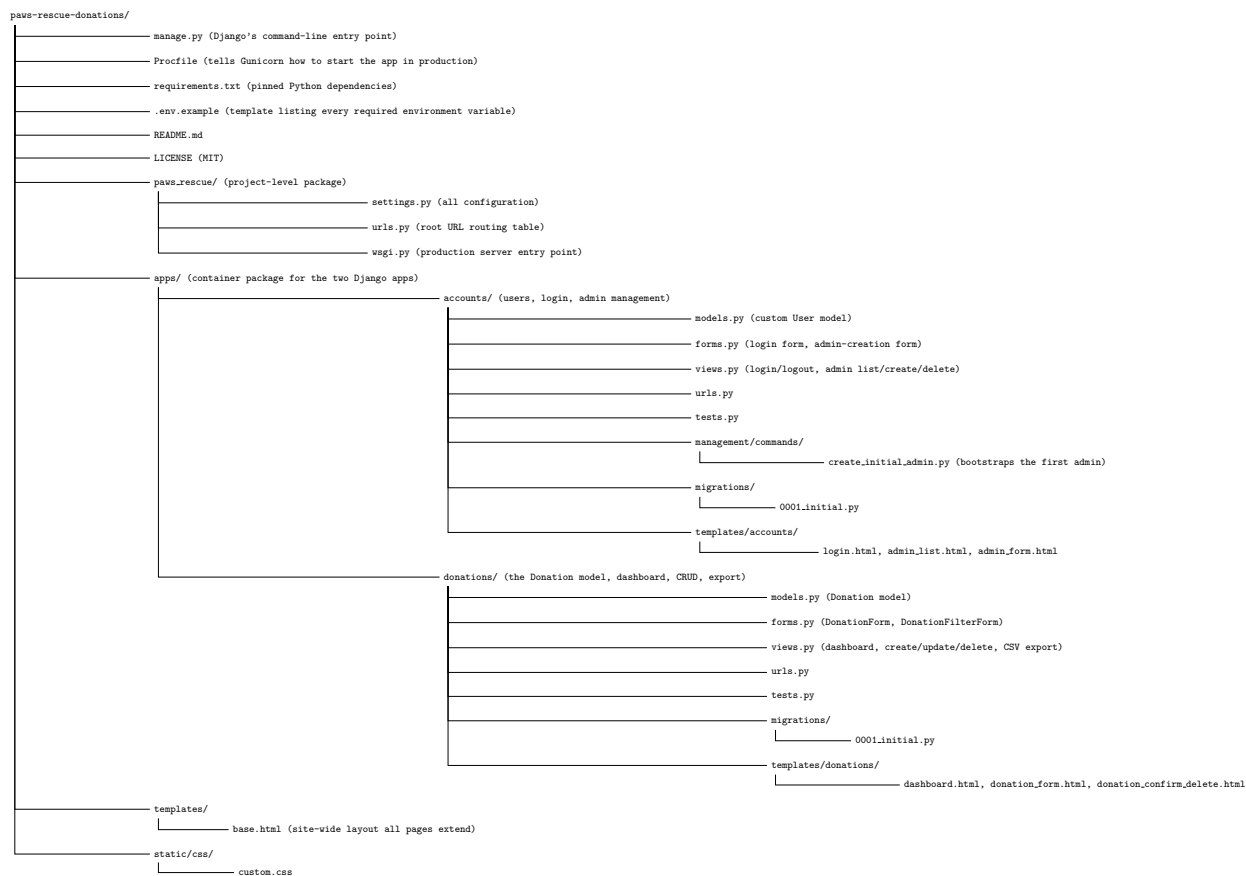


Figure 1: Every file in the repository, grouped by directory.

3 Architecture Overview

WSGI (Web Server Gateway Interface). A standard Python interface that lets any WSGI-compatible web server (like Gunicorn) hand incoming HTTP requests to any WSGI-compatible Python framework (like Django) in a uniform way. `paws_rescue/wsgi.py` exposes the `application` callable that Gunicorn calls for every request.

Gunicorn. “Green Unicorn” – a production-grade WSGI server for Python. Django’s own built-in server (`manage.py runserver`) is only meant for local development; Gunicorn is what actually serves traffic once deployed. The `Procfile` in this project runs `gunicorn paws_rescue.wsgi`.

WhiteNoise. A library that lets a Python web app serve its own static files (CSS, JavaScript, images) directly, without needing a separate web server like Nginx in front of it. It is added as Django middleware (`whitenoise.middleware.WhiteNoiseMiddleware`) and is what makes the free-tier deployment story simple: one process does everything.

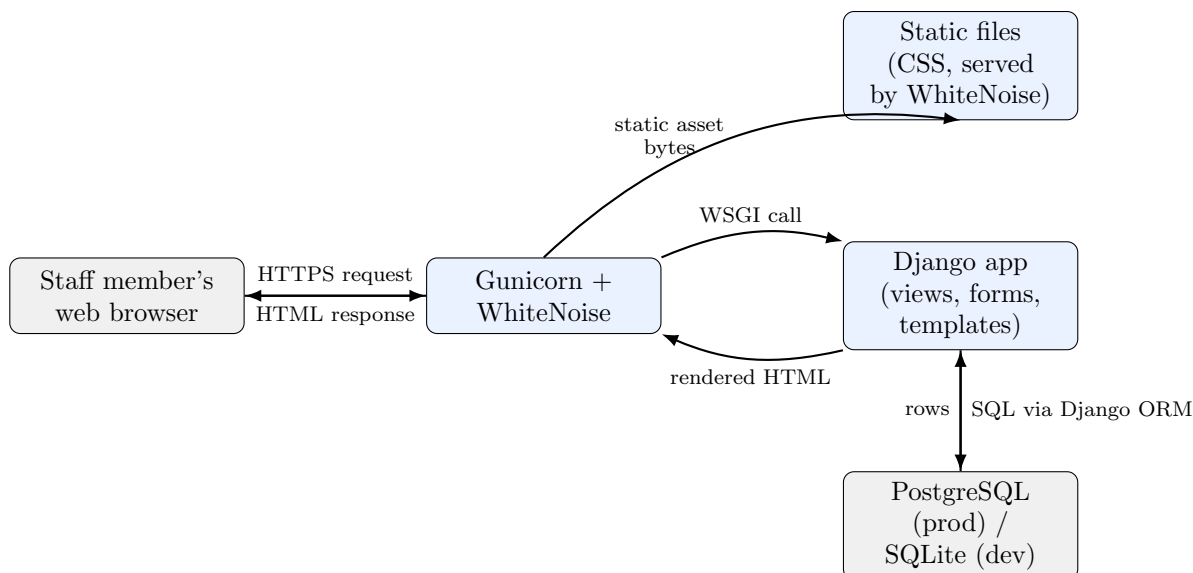


Figure 2: Component-level architecture. A single Gunicorn process runs the WSGI application; WhiteNoise serves CSS directly from that same process instead of needing a separate static file server; Django talks to whichever database `DATABASE_URL` points at.

ORM (Object-Relational Mapper). A layer that lets you write and query database rows using Python objects and method calls (`Donation.objects.filter(...)`) instead of writing raw SQL by hand. Django’s ORM translates these calls into SQL statements appropriate for whichever database engine is configured.

4 Request Lifecycle and URL Routing

Every incoming HTTP request follows the same path through Django before a response goes back to the browser. The diagram below traces a concrete example: a staff member visiting `/dashboard/`.

HTTP request/response. The basic unit of web communication. A browser sends a *request* (a method like GET or POST, a URL, and optional form data); the server sends back a *response* (a status code like 200 or 302, and a body – usually HTML).

Middleware. A chain of small processing steps that every request (and every response) passes through before/after reaching the view – e.g. attaching the current session, checking CSRF tokens, or attaching the logged-in user object. Configured as an ordered list, `MIDDLEWARE`, in `settings.py`; order matters because each layer wraps the next.

Decorator. A Python function that wraps another function to add behaviour without changing its body. `@login_required` wraps every protected view so it redirects anonymous users to the login page before the view’s own code ever runs; `@user_passes_test(is_admin, login_url='login')` does the same but checks a custom predicate (here, “is this user staff”).

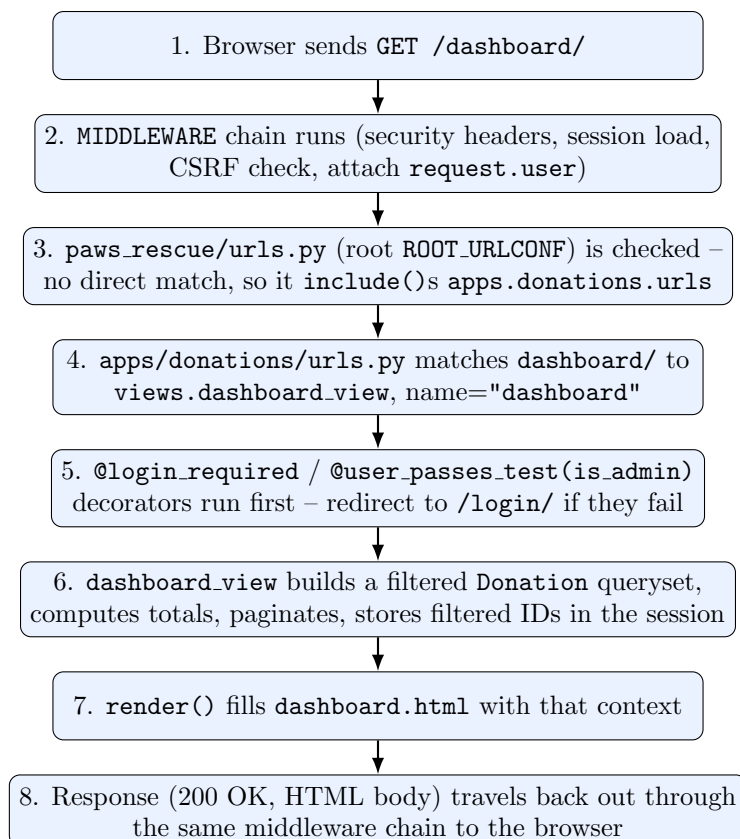


Figure 3: Request lifecycle for GET /dashboard/.

4.1 Full URL Table

Path	View	Name
/	redirects to dashboard	home
/admin/	Django's built-in admin site	<code>django.contrib.admin</code>
/login/	<code>accounts.views.login_view</code>	login
/logout/	<code>accounts.views.logout_view</code>	logout
/admins/	<code>accounts.views.admin_list_view</code>	admin_list
/admins/create/	<code>accounts.views.admin_create_view</code>	admin_create
/admins/<uuid>/delete/	<code>accounts.views.admin_delete_view</code>	admin_delete
/dashboard/	<code>donations.views.dashboard_view</code>	dashboard
/donations/create/	<code>donations.views.donation_create_view</code>	donation_create
/donations/<int:pk>/edit/	<code>donations.views.donation_update_view</code>	donation_update
/donations/<int:pk>/delete/	<code>donations.views.donation_delete_view</code>	donation_delete
/donations/export/	<code>donations.views.export_csv_view</code>	export_csv

Note the root `urls.py` (`paws_rescue/urls.py`) itself only declares four patterns – `admin/`, the home redirect, and two `include()` calls that hand off all other matching to each app's own `urls.py`. Django tries patterns top to bottom and stops at the first match, so app ordering in `urls.py` can matter if two apps ever defined overlapping paths (they do not, here).

UUID (Universally Unique Identifier). A 128-bit value, usually written as 32 hex digits in five dash-separated groups (e.g. `3fa85f64-5717-...`), generated so that two independently created ones are astronomically unlikely to collide. `User.id` is a UUID rather than a simple auto-incrementing integer (see §5), which is why the admin-delete URL uses the `<uuid:user_id>` path converter while donation URLs use plain `<int:pk>`.

5 Data Models and Entity Relationships

Model. A Python class that Django maps onto a database table: each class attribute becomes a column, and each class instance becomes a row. Django auto-generates the SQL to create, alter, and query that table from the model definition.

Migration. An auto-generated (or hand-written) Python file that describes one incremental change to the database schema – e.g. “create this table with these columns.” Migrations are Django’s version-control system for the database: running `manage.py migrate` applies any migrations that haven’t been applied yet, in dependency order. Every model change should be accompanied by a new migration file; this project has exactly one migration per app, both named `0001_initial.py`, meaning the schema has not changed since the apps were first created.

Primary key. The column (or combination of columns) that uniquely identifies each row in a table. `Donation` uses Django’s default auto-incrementing `BigAutoField`; the custom `User` model instead uses a UUID (see below) as its primary key.

Foreign key. A column in one table that stores the primary key of a row in another table, creating a link between them (e.g. “this donation belongs to this pet” or “this order belongs to this customer”). **This project’s `Donation` model has no foreign key to `User`** – there is no column recording which admin entered a given donation. This is a design gap the project’s own README explicitly flags under “What I’d do differently”: “Record who entered each donation. `Donation` has no foreign key to `User`, so there’s no audit trail, and deletes are hard deletes.”

6 The accounts App

This app owns everything related to *who* can use the system: the custom user model, the login form and view, and the screens for managing which email addresses have admin access.

6.1 Custom User Model (`accounts/models.py`)

Django ships a built-in `User` model that authenticates by `username`. This project replaces it entirely because there should be no usernames – only email addresses – and no public self-signup.

`AbstractBaseUser` / `PermissionsMixin`. Two building blocks Django provides for writing a fully custom user model. `AbstractBaseUser` supplies password hashing and session-handling machinery without assuming anything about the identifying field. `PermissionsMixin` adds

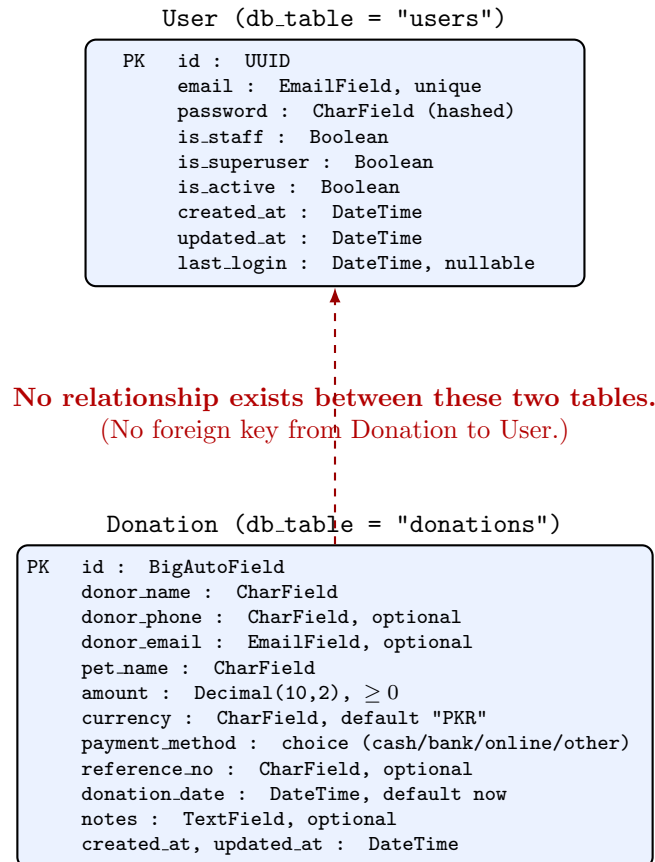


Figure 4: Entity-relationship diagram. Both models were verified to be fully independent – there is no `ForeignKey`, `OneToOneField`, or `ManyToManyField` connecting `Donation` to `User` anywhere in the codebase. `User` also gains `groups` and `user_permissions` many-to-many fields for free from Django’s `PermissionsMixin`, connecting it to Django’s built-in `auth.Group` and `auth.Permission` tables (not drawn, as the app does not use groups or fine-grained permissions in its own logic).

the `is_superuser`, `groups`, and `user_permissions` fields used by Django’s permission system. Combining both, as `User(AbstractBaseUser, PermissionsMixin)` does here, gives a model that looks and behaves like Django’s default user but with whatever fields the developer chooses.

```

class User(AbstractBaseUser, PermissionsMixin):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    email = models.EmailField(unique=True, max_length=255)
    is_staff = models.BooleanField(default=False)
    is_superuser = models.BooleanField(default=False)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(default=timezone.now)
    updated_at = models.DateTimeField(auto_now=True)

    objects = UserManager()
    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = []

```


USERNAME_FIELD. The Django setting on a custom user model that tells the authentication system which field acts as the unique login identifier. Setting it to 'email' means Django's `authenticate()` and login machinery expect an email address wherever they would normally expect a username.

`AUTH_USER_MODEL = 'accounts.User'` in `settings.py` tells Django to use this model everywhere (admin site, `request.user`, permissions) instead of its default. This has to be set before the very first migration is run, because swapping the user model later requires rebuilding the whole migration history – the README calls this out explicitly as a “decision to make on day one.”

Manager. The interface used to run table-wide queries, e.g. `User.objects.filter(...)`. Every model gets a default manager (`objects`) automatically; a custom manager subclass is only required when the default behaviour needs to change – here, because `create_user/create_superuser` must accept `email` instead of `username`.

`UserManager` reimplements both creation methods:

```
def create_user(self, email, password=None, **extra_fields):
    if not email:
        raise ValueError('The Email field must be set')
    email = self.normalize_email(email)
    user = self.model(email=email, **extra_fields)
    user.set_password(password)
    user.save(using=self._db)
    return user
```

`set_password()` runs the password through Django's configured hashing algorithm before it is ever written to the database – plaintext passwords are never stored. `create_superuser` additionally forces `is_staff=True` and `is_superuser=True`, raising a `ValueError` if a caller tries to override either to `False`.

One documented consequence: Django's test client and `login()` helper still expect the keyword argument `username=`, even though this project has no username field. The project's own tests pass the email through that keyword (`self.client.login(username='admin@test.com', password=...)`) – this is noted explicitly in the README as “the catch.”

6.2 Login Form and View

`EmailAuthenticationForm` (in `accounts/forms.py`) is a plain `forms.Form` (not tied directly to a model) with `email` and `password` fields. Its `clean()` method calls Django's `authenticate(request, username=email, password=password)` – note `username=email` again, translating from Django's generic authentication API to this project's email-only model. If authentication fails, a `ValidationError` with code `invalid_login` is raised so the template can show “Invalid email or password.”

CSRF (Cross-Site Request Forgery) protection. An attack where a malicious site tricks a logged-in user's browser into submitting a request to a different site (e.g. this app) using the victim's own cookies, without their consent. Django defends against it by embedding a secret, per-session token in every form (`{% csrf_token %}` in the templates) and rejecting POST requests that don't include a matching token. This is enforced by `django.middleware.csrf.CsrfViewMiddleware` in the `MIDDLEWARE` list. Every form template

in this project (login, admin creation, donation create/edit, delete confirmation, logout) includes `{% csrf_token %}`.

`login.view` (in `accounts/views.py`) is deliberately stricter than just “authenticate successfully”: even a validly authenticated user is only actually logged in and redirected if `user.is_staff` is true

```
if user and user.is_staff:
    login(request, user)
    ...
    return redirect('dashboard')
else:
    messages.error(request, 'You do not have permission to access this application.')
```

This means a valid `User` row that is not staff (there is no code path that currently creates one, since both `create_user` paths used in this app force `is_staff=True`) would still be refused entry to the app even after a correct password.

Django messages framework. A built-in mechanism for queueing one-off notifications (“Donation created successfully!”) that survive exactly one redirect-and-render cycle, then disappear. `messages.success(...)`, `messages.error(...)`, and `messages.info(...)` are used throughout this project’s views; `base.html` renders them as dismissible Bootstrap alerts.

6.3 Admin Management Views

Three views, all gated by `@login_required` and `@user_passes_test(is_admin, login_url='login')`:

- `admin_list.view` – lists every `User` with `is_staff=True`, newest first, and flags whichever row matches the currently logged-in user’s ID (`current_user_id`) so the template can hide the delete button for your own row.
- `admin_create.view` – renders/handles `AdminCreationForm`, which is a `ModelForm` bound to `User` with only `email` and `password` exposed. Its overridden `save()` sets the password via `set_password()` and forces `is_staff = True` and `is_superuser = True` on every admin created through this screen – so every admin created via the web UI is, in Django’s own permission terms, a superuser.
- `admin_delete.view` – POST-only (enforced by `@require_http_methods(["POST"])`), looks the target user up with `get_object_or_404`, and explicitly refuses to proceed if `user_to_delete.id == request.user.id`, redirecting back with an error message instead. This is a hard-deleted row (`user_to_delete.delete()`) – there is no soft-delete / deactivation flag used here despite `is_active` existing on the model.

ModelForm. A Django form class auto-generated from a model, saving the developer from re-declaring every field by hand and giving a `.save()` method that writes straight to the database. `AdminCreationForm` and `DonationForm` (see §7.2) are both `ModelForms`.

6.4 Templates

`login.html`, `admin_list.html`, and `admin_form.html` all `{% extends 'base.html' %}` and only override the `title` and `content` blocks (see §7.6 for how template inheritance works). `admin_list.html`

renders a Bootstrap table of admins with a `Delete` button wrapped in a JavaScript `confirm()` dialog and hidden entirely for the current user’s own row (`{% if admin.id != current_user.id %}`).

6.5 Tests (`accounts/tests.py`)

Four tests, all using Django’s `TestCase` (which wraps each test in a database transaction that is rolled back afterwards, so tests never see each other’s data):

1. `test_unauthorized_user_redirected_to_login` – an anonymous GET to `/dashboard/` returns a 302 redirect whose target contains `/login/`.
2. `test_admin_can_login` – posting valid credentials to `/login/` returns a 302 (redirect to dashboard on success).
3. `test_admin_list_requires_login` – confirms the 302-then-200 pattern: unauthenticated request redirects, then after `client.login(username=..., password=...)` the same URL returns 200.
4. `test_cannot_delete_self` – posts to the delete endpoint with the logged-in user’s own ID and asserts the user still exists afterwards.

7 The donations App

This is the core of the application: the `Donation` model, the dashboard with its filters and analytics, the create/edit/delete CRUD flow, and CSV export.

7.1 The Donation Model

```
class Donation(models.Model):
    PAYMENT_METHOD_CHOICES = [
        ('cash', 'Cash'), ('bank', 'Bank Transfer'),
        ('online', 'Online Payment'), ('other', 'Other'),
    ]
    donor_name = models.CharField(max_length=255)
    donor_phone = models.CharField(max_length=50, blank=True, null=True)
    donor_email = models.EmailField(blank=True, null=True)
    pet_name = models.CharField(max_length=255, help_text='...')
    amount = models.DecimalField(max_digits=10, decimal_places=2,
                                validators=[MinValueValidator(Decimal('0.00'))])
    currency = models.CharField(max_length=10, default='PKR')
    payment_method = models.CharField(max_length=20,
                                     choices=PAYMENT_METHOD_CHOICES, default='cash')
    reference_no = models.CharField(max_length=255, blank=True, null=True)
    donation_date = models.DateTimeField(default=timezone.now)
    notes = models.TextField(blank=True, null=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'donations'
        ordering = ['-donation_date']
```

Points worth calling out:

- `amount` uses `DecimalField`, not a float, because floating-point binary representation cannot exactly represent most decimal currency values (e.g. $0.1 + 0.2$ is not exactly 0.3 in binary floating point) – `Decimal` avoids that class of rounding bug entirely.
- `currency` is a free-text `CharField` with no choice list, so nothing stops an operator typing "pkr" or "Rs". The README lists this as a known gap: “Validate currency against a choice list. It’s a free-text field defaulting to PKR, so typos ... would silently split the totals.” The filtering code does use `currency__iexact` to at least make the *filter* case-insensitive (see below), but the stored value itself is never normalised.
- `Meta.ordering = ['-donation_date']` is the model’s default sort whenever no explicit `.order_by()` is given, newest first.
- There is no soft-delete flag and no foreign key to the `User` who entered the record – both explicitly called out as gaps in the README’s “What I’d do differently” section (see §5).

7.2 Forms (donations/forms.py)

`DonationForm` is a `ModelForm` covering every field except the auto-managed timestamps. It overrides `donation_date` to be optional (`required=False`) with a custom `clean_donation_date` that fills in `timezone.now()` if the operator leaves it blank – so the field behaves like an editable-but-optional timestamp rather than a strict requirement. It also defines `clean_amount` to reject negative values (a second line of defense on top of the model-level `MinValueValidator`).

`clean_<field>` methods. Django’s per-field validation hook convention: a method named `clean_amount` on a form automatically runs during `form.is_valid()` and can transform or reject that one field’s value before the whole form is considered valid.

`DonationFilterForm` is a plain (non-model) form with seven optional fields: free-text `search`, `date_from/date_to`, `payment_method` (a dropdown seeded from the model’s own `PAYMENT_METHOD_CHOICES` plus an “All” option), `currency`, `amount_min/amount_max`, and `sort_by`. None of its fields are required – an empty query string is a completely valid, unfiltered request. The “Amount From”/“Amount To” labels were deliberately chosen over “Min Amount”/“Max Amount”; the README explains why: a test asserting a page did *not* contain the text “Max” (checking that pet “Max” wasn’t shown) was accidentally matching the form label “Max Amount” instead, a false positive fixed by renaming the label.

7.3 The Dashboard View: Filtering, Analytics, Pagination

`dashboard_view` is the largest and most consequential piece of logic in the project. Its flow:

Queryset. A lazily-evaluated representation of a database query built by chaining Django ORM methods (`.filter()`, `.order_by()`, `.values()`, ...). No SQL actually runs until the queryset is iterated, counted, or otherwise “consumed” – which is why the same `donations` queryset can be filtered step by step across many `if` blocks before a single query is issued.

Aggregation (Sum, Count). ORM functions that compute a value across many rows at the database level (e.g. “sum this column across all matching rows”) rather than pulling every row into Python and adding them up by hand – far more efficient for large tables.

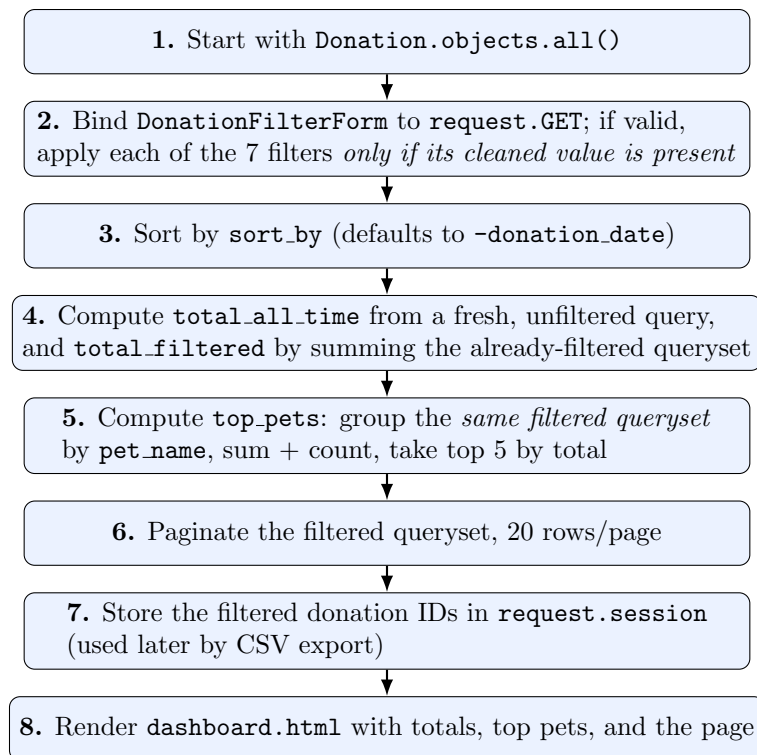


Figure 5: Data flow inside `dashboard_view` for every dashboard page load.

Two design decisions here are explicitly documented in the README as bugs that were found and fixed during development:

- **Top-pets panel used to ignore filters.** It originally aggregated over `Donation.objects.all()`, so it disagreed with the filtered total shown right next to it on the same page. The fix (visible in the current code) aggregates over the `donations` queryset *after* filters have already been applied to it.
- **Amount bounds use an explicit is not None check**, not a plain truthiness check (`if amount_min:`), specifically so that a legitimate filter value of 0 is not silently treated as “no filter” – 0 is falsy in Python but a meaningful minimum amount.

Pagination. Splitting a long result set into fixed-size pages so a single request doesn’t have to render (and the browser doesn’t have to download) every row at once. Django’s `Paginator(donations, 20)` slices the queryset into pages of 20; the template reads `page_obj.has_next/has_previous` to build First/Previous/Next/Last links that also carry forward every current filter as query parameters.

7.4 CSV Export and the Session-Based Filter Handoff

CSV (Comma-Separated Values). A plain-text tabular file format, one row per line and commas separating columns, that spreadsheet programs (Excel, Google Sheets) can open directly.

Session. A server-side store of small pieces of data tied to a particular browser via a cookie holding an opaque session ID. Django persists session data in the database by default; `request.session[key] = value` writes to it, `request.session.get(key)` reads it back on a later request from the same browser.

The CSV export link on the dashboard is a plain `<a href="% url 'export_csv' %">` – an ordinary GET request carrying no query string of its own, and therefore no way to know which filters were active when the user clicked it. The project’s chosen solution: every dashboard page load writes the current filtered donation IDs into the session (`request.session['filtered_donation_ids'] = list(donations.values_list('id', flat=True))`), and `export_csv_view` reads that same session key back to decide which rows to include.

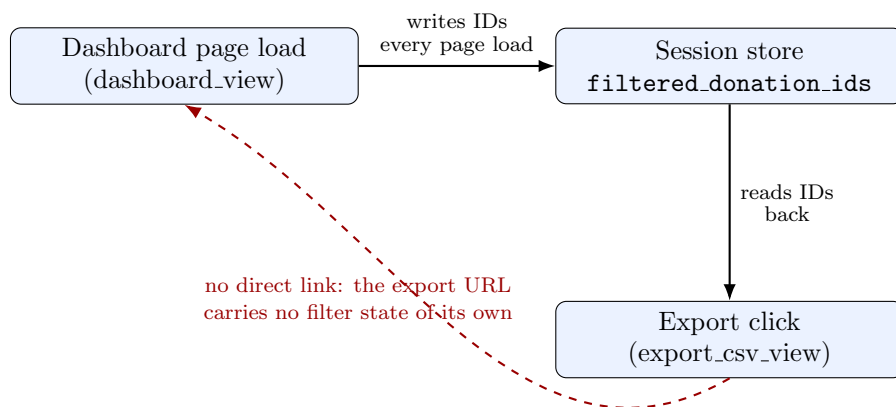


Figure 6: The session hand-off between dashboard and export. The README names this explicitly as “the weakest part of the design”: the session grows with the table over time, can go stale if data changes between page load and export click, and (per “What I’d do differently”) the recommended fix is to have the export view re-apply the filter form to the query string directly instead of depending on session state.

The export itself uses Python’s `csv.writer` with `quoting=csv.QUOTE_ALL` (every field quoted, avoiding ambiguity if any text field contains a comma) and streams a fixed 12-column header (ID, Donor Name, Donor Phone, Donor Email, Pet Name, Amount, Currency, Payment Method, Reference No, Donation Date, Notes, Created At) via an `HttpResponse` with `Content-Type: text/csv` and a `Content-Disposition: attachment; filename="donations_YYYY-MM-DD.csv"` header that makes the browser download it as a file rather than display it inline.

7.5 Donation Create / Update / Delete Views

`donation_create_view` and `donation_update_view` share the same template (`donation_form.html`) and the same pattern – unbound form on `GET`, bind + validate + save on `POST`, redirect to the dashboard with a success message, or re-render with field errors collected into the messages framework on failure. `donation_update_view` additionally fetches the existing row with `get_object_or_404(Donation, pk=pk)` and passes `instance=donation` into the form so Django knows to `UPDATE` rather than `INSERT`.

get_object_or_404. A Django shortcut that looks a row up by its primary key (or other filter) and automatically returns an HTTP 404 (Not Found) response instead of raising an unhandled exception if no matching row exists.

`donation_delete_view` renders a confirmation page (`donation_confirm_delete.html`) on `GET` and only actually deletes the row on `POST` – this is a genuine hard delete (`donation.delete()`), with no undo and no soft-delete flag on the model.

7.6 Templates and Template Inheritance

Template inheritance. A mechanism where a base HTML file (`templates/base.html` here) defines the overall page skeleton – head, nav bar, message area, footer, script tags – with named `{% block %}` placeholders, and every other template does `{% extends 'base.html' %}` and only fills in the blocks it cares about (usually just `title` and `content`). This avoids repeating the navigation bar and `<head>` boilerplate in every single page.

`base.html` pulls in Bootstrap 5 and Bootstrap Icons from a public CDN (content delivery network) and links the project’s one local stylesheet, `static/css/custom.css`, via `{% load static %}` / `{% static 'css/custom.css' %}`. The navigation bar (Dashboard, Add Donation, Admins links, plus the logged-in user’s email and a logout button) is only rendered when `user.is_authenticated` – so the login page itself has no nav bar. Messages queued via the Django messages framework render as dismissible Bootstrap alerts directly under the nav bar on every page.

`dashboard.html` is the most complex template: three analytics cards (all-time total, filtered total, top-5-pets list), a filter sidebar built directly from `filter_form` field-by-field (rather than a generic `{{ form }}` loop, so each field can sit next to its own label and spacing), a donations table, and pagination links that manually rebuild the current query string (`{% for key, value in request.GET.items %}...{% endfor %}`) so that navigating to page 2 doesn’t silently drop whatever filters were active.

`donation_form.html` is shared by both create and update, driven by a `title` context variable (“Add New Donation” / “Edit Donation”) and an optional `donation` variable used only by the update path.

7.7 Tests (donations/tests.py)

Seven tests, sharing a `setUp` that creates one superuser and logs in, plus two sample donations (John Doe / Max / 1000 PKR / cash, Jane Smith / Bella / 500 PKR / online):

1. `test_admin_can_access_dashboard` – dashboard returns 200 and contains the heading text “Donation Dashboard”.

2. `test_admin_can_create_donation` – posts a new donation, checks the redirect, the row count increment, and the saved field values.
3. `test_csv_export_returns_csv_response` – checks `Content-Type`, the `attachment` disposition, the filename prefix, and that both the header row and a known donor’s name appear in the exported bytes.
4. `test_search_by_donor_name / test_search_by_pet_name` – confirm the `Q(donor_name__icontains=...) | Q(pet_name__icontains=...)` search matches one donation and excludes the other.
5. `test_filter_by_payment_method` – confirms a `payment_method` filter still shows the matching donation.
6. `test_amount_validation` – posts a negative amount and asserts no row was created, exercising both the model-level `MinValueValidator` and the form’s own `clean_amount`.

Together with the four `accounts` tests, the project totals **eleven** tests, matching the count the README states, run with `python manage.py test`.

8 The Management Command: `create_initial_admin`

Management command. A custom subcommand added to `manage.py`, invoked as `python manage.py <command_name>`. Django auto-discovers any `Command` class placed in an app’s `management/commands/<name>.py` file. This is the standard way to add one-off or recurring administrative scripts that need full access to the Django ORM and settings.

`create_initial_admin.py` reads `INITIAL_ADMIN_EMAIL` and `INITIAL_ADMIN_PASSWORD` from the environment; if either is missing, it prints a warning and exits without error. If a `User` with that email already exists, it prints a warning and skips (idempotent – safe to run more than once). If neither of those hold, it calls `User.objects.create_superuser(...)` and prints a success message. This is meant to be run exactly once, right after the very first deploy, to bootstrap the first login before any admin exists to create others through the web UI.

9 Configuration: `settings.py`

Environment variable. A named value set outside the program itself (in the shell, a hosting platform’s dashboard, or a local `.env` file) and read at runtime – the standard way to keep secrets and per-environment values (database URLs, debug flags) out of source code.

`settings.py` calls `load_dotenv()` at import time (from `python-dotenv`), so a local `.env` file’s contents behave exactly like real environment variables during development, while a real deployment platform would set the same variable names directly.

Setting	What it controls
SECRET_KEY	Cryptographic key used for signing sessions, CSRF tokens, etc. Falls back to an obviously-insecure placeholder string if unset, so the app runs locally without any setup but should never be deployed with the default.
DEBUG	When true, Django shows detailed error pages with stack traces and disables several production security features. Must be False in production.
ALLOWED_HOSTS	Comma-separated list of hostnames Django will accept requests for – a defense against HTTP Host-header attacks.
TESTING	Not loaded from the environment – computed as <code>'test' in sys.argv</code> , i.e. true only when running under <code>manage.py test</code> . Used to skip production-only security settings and static storage during test runs (see below).
DATABASE_URL	If set, parsed by <code>django.utils.functional</code> into a PostgreSQL connection; if empty, the app falls back to a local SQLite file (<code>db.sqlite3</code>).
AUTH_USER_MODEL	Points Django at the custom <code>accounts.User</code> model instead of its built-in one.
STORAGES["staticfiles"]	<code>ManifestStaticFilesStorage</code> (WhiteNoise's manifest-based, hashed-filename, gzip/brotli-compressed storage) in normal operation; swapped to plain <code>StaticFilesStorage</code> when TESTING is true, because the manifest storage requires <code>collectstatic</code> to have already run, which the test runner never does.
LOGIN_URL / LOGIN_REDIRECT_URL / LOGOUT_REDIRECT_URL	Where <code>@login_required</code> redirects unauthenticated users, and where successful login/logout send the user afterwards.
SESSION_COOKIE_AGE / SESSION_SAVE_EVERY_REQUEST	Sessions expire after 86400 seconds (1 day); the session is re-saved (extending its expiry and re-writing its data, including the user-ID list) on <i>every</i> request.
Production security block	Only applied when not DEBUG and not TESTING : forces HTTPS redirects, secure (HTTPS-only) cookies, XSS filter header, content-type sniffing protection, and <code>X-Frame-Options: DENY</code> (blocks the site from being embedded in an <code><iframe></code> on another site, a defense against clickjacking).

The TESTING flag, explained. This is the fix for a real bug the README documents: with `DEBUG=False`, `SECURE_SSL_REDIRECT` turned every test-client request into an HTTP 301 redirect to https (which Django's plain-HTTP test client can't follow the way a real browser would), and the manifest static storage raised errors because `collectstatic` never runs before tests. `manage.py test` does *not* set `DEBUG=True`, so a setting gated only on **not DEBUG** silently misfires under the test runner too. Gating on an explicit `'test' in sys.argv` check fixes both problems by treating “running tests” as its own third mode, distinct from both dev and prod.

10 Dependencies (requirements.txt)

Package (pinned version)	Role
Django==4.2.11	The web framework itself.
psycopg2-binary==2.9.11	PostgreSQL database driver – lets Python/Django talk to a PostgreSQL server. Only exercised when DATABASE_URL is set; local development with SQLite doesn't need it, but it's installed unconditionally so the same environment works in both modes.
python-dotenv==1.0.1	Loads key/value pairs from a .env file into the process environment, so local development can use a file instead of exporting shell variables.
gunicorn==21.2.0	Production WSGI server (see §3).
whitenoise==6.6.0	Serves static files directly from the Django/Gunicorn process (see §3).
dj-database-url==2.1.0	Parses a single DATABASE_URL connection string (the format most hosting platforms provide) into Django's DATABASES dictionary format.

Pinned version. Specifying an exact package version (==4.2.11 rather than >=4.2) so that every install of requirements.txt produces the same dependency versions, avoiding “it works on my machine” drift between environments.

11 Deployment (Procfile)

```
web: gunicorn paws_rescue.wsgi --log-file -
```

Procfile. A convention (originating with Heroku, also understood by Render and Railway) for declaring the command(s) a hosting platform should run to start the app. The **web:** line here tells the platform to run Gunicorn against the project's WSGI application, logging to stdout (--log-file -) so the platform's own log viewer can capture it.

Per the README, the full deployment recipe is: build step `pip install -r requirements.txt` && `python manage.py collectstatic --noinput` && `python manage.py migrate`, then the Procfile's start command, with SECRET_KEY, DEBUG=False, ALLOWED_HOSTS, DATABASE_URL, INITIAL_ADMIN_EMAIL, and INITIAL_ADMIN_PASSWORD all set as environment variables on the host, and `create_initial_admin` run once by hand after the first deploy.

collectstatic. A management command that gathers every app's static files into the single STATIC_ROOT directory (here, BASE_DIR / 'staticfiles'), applying whatever storage backend is configured (WhiteNoise's manifest storage, in production) – generating the hashed filenames and compressed variants that `CompressedManifestStaticFilesStorage` needs at request time.

12 Static Assets

The only local static asset is `static/css/custom.css`: layout helpers to pin the footer to the bottom of the viewport on short pages, subtle hover transitions on cards and analytics tiles, and a small responsive breakpoint that shrinks table font size and button padding under 768px width. Everything else visual (Bootstrap 5, Bootstrap Icons) is loaded from a public CDN in `base.html` rather than vendored into the repository.

13 Environment Variable Surface (`.env.example`)

Variable	Purpose
<code>SECRET_KEY</code>	Random 50-character secret; the comment shows the exact Django command to generate one.
<code>DEBUG</code>	<code>True</code> for local development in the example file.
<code>ALLOWED_HOSTS</code>	Defaults to <code>localhost,127.0.0.1</code> .
<code>DATABASE_URL</code>	Left empty in the example – falls back to SQLite.
<code>INITIAL_ADMIN_EMAIL</code> /	Consumed only by the <code>create_initial_admin</code> management command.
<code>INITIAL_ADMIN_PASSWORD</code>	

14 Known Limitations (as documented by the project itself)

The following are not this document’s own critique – they are the exact self-assessment written into the project’s README under “Challenges,” “What I learned,” and “What I’d do differently,” reproduced here because they are genuinely useful interview-defense material and because inventing a different set of limitations would violate this document’s no-fabrication rule.

- Session-based CSV export is called out as “the weakest part of the design”: it grows unbounded with the table, and can go stale between page load and export click. Recommended fix: re-derive the filtered queryset from the query string inside the export view instead.
- Filtering logic is duplicated field-by-field in `dashboard_view` with no shared helper, and the export view cannot reuse it. Recommended fix: adopt `django-filter` or extract a shared filtering helper.
- Tests assert on raw page text (`assertContains/assertNotContains`), which is fragile – demonstrated by the “Max Amount” label collision described in §8.2. Recommended fix: assert on the queryset or specific structured data instead.
- Production security settings are gated by a combination of `DEBUG` and an ad hoc `'test'` in `sys.argv` check rather than separate base/dev/prod settings modules.
- No database indexes on `donation_date` or `pet_name`, despite both driving the default ordering and the top-pets aggregation.
- No foreign key from `Donation` to `User` – no audit trail of who entered each donation, and deletes are hard deletes with no history.
- `currency` is unvalidated free text, so inconsistent casing or typos (“pkr”, “Rs”) would silently split totals that should be combined.

Appendix: Glossary

App

A self-contained module of a Django project (accounts, donations).

CSRF

Cross-Site Request Forgery; Django's defense is a per-session token embedded in every form.

CSV

Comma-Separated Values, a plain-text spreadsheet interchange format.

Decorator

A function wrapper that adds behaviour (e.g. auth checks) without changing the wrapped function's body.

Foreign key

A column linking one table's row to another table's row; absent between `Donation` and `User` in this project.

Framework

Pre-built application scaffolding (Django) that supplies common web app machinery.

Gunicorn

Production-grade Python WSGI server.

Management command

A custom `manage.py` subcommand.

Middleware

Ordered request/response processing steps that wrap every view.

Migration

A versioned, incremental database schema change file.

Model

A Python class mapped to a database table.

ModelForm

A Django form auto-generated from a model.

MVT

Django's Model-View-Template pattern (View $\approx$ Controller, Template $\approx$ View, in classic MVC terms).

ORM

Object-Relational Mapper; lets Python code generate SQL.

Pagination

Splitting a long result set into fixed-size pages.

Primary key

The column that uniquely identifies each table row.

Queryset

A lazily-evaluated, chainable database query object.

Session

Server-side, per-browser data store keyed by a cookie.

Static file

A CSS/JS/image file served as-is, not generated per request.

UUID

A 128-bit identifier, astronomically unlikely to collide.

WhiteNoise

Library that serves static files from the app process itself.

WSGI

The standard interface between Python web servers and frameworks.